BPSC TRE 4.0

Computer Science (Class 11–12)

Day 4

1. Computer Fundamentals > Operating Systems

2026-10-31

BPSC TRE 4.0 Computer Science — Day 4 — 1. Computer Fundamentals > Operating Systems — 2026-10-31

Notes • 47 Questions • Answer Key + व्याख्या

OS Part 2/4 — CPU Scheduling

① पढ़ने से पहले

- आज का Part 2/4: CPU (Central Processing Unit) scheduling, process states, critical section, semaphore, deadlock, memory management, virtual memory, disk scheduling और UNIX/Windows के कुछ facts.
- Part 1 recap: OS (Operating System) = computer का मैनेजर; process = चलता हुआ program; thread = process का छोटा हिस्सा.

② 1. आसान भाषा में

- एक CPU एक समय में एक ही process चला सकता है. बाकी processes Ready queue में इंतज़ार करते हैं. OS तय करता है कि अगला कौन चलेगा. इस फ़ैसले की विधि को CPU Scheduling कहते हैं.
- Process के states: New (बना), Ready (CPU का इंतज़ार), Running (CPU पर), Blocked/Waiting (I/O का इंतज़ार), Terminated (खत्म).
  - →Ready→Running = dispatch (CPU मिला)
  - →Running→Ready = preempt (time खत्म या बड़ा priority आया)
  - →Running→Blocked = process खुद I/O माँगे
  - →Blocked→Ready = I/O पूरा हुआ

③ 2. रोज़मर्रा का example

- बैंक की एक खिड़की (CPU) और बाहर लाइन (Ready queue) सोचो.
  - →जो पहले आया उसे पहले = FCFS (First Come First Serve)
  - →जिसका काम सबसे छोटा हो उसे पहले = SJF (Shortest Job First)
  - →हर किसी को 2 मिनट, फिर लाइन के पीछे = RR (Round Robin)
  - →VIP को पहले = Priority

④ 3. हर algorithm step by step

- Burst time = process को CPU पर कितना समय चाहिए. Arrival time = कब आया.

- Turn Around Time (TAT) = Completion − Arrival. Waiting Time (WT) = TAT − Burst.
- FCFS: आने के क्रम में, non-preemptive. Convoy effect (एक बड़ा job सबको रोके).
- SJF: सबसे छोटा burst पहले; average waiting time सबसे कम; पर लंबे jobs को starvation. Preemptive रूप = SRTF (Shortest Remaining Time First).
- RR: हर process को time quantum (जैसे 2 ms), फिर पीछे. Preemptive. Quantum बहुत बड़ा = FCFS जैसा; बहुत छोटा = context switch ज़्यादा.
- Priority: ऊँचा priority पहले. Starvation का इलाज aging (इंतज़ार बढ़े तो priority बढ़ाओ).
- MLFQ (Multilevel Feedback Queue): कई queues; process queue बदल सकता है. SPN (Shortest Process Next) में अगला burst exponential smoothing से अंदाज़ा.
- Linux: Fair preemptive scheduler (CFS — Completely Fair Scheduler).

⑤ Worked example (RR)

- P1 = 5, P2 = 3, P3 = 1 (सब time 0 पर), quantum = 2. TAT और average निकालो.

```
P1(0-2) P2(2-4) P3(4-5 done)
P1(5-7) P2(7-8 done) P1(8-9 done)
Completion: P1=9, P2=8, P3=5
Avg TAT = (9+8+5)/3 = 7.33 ms
```

⑥ 4. Exam Point + Trap

- Context switch non-preemptive SJF में बीच में नहीं होता (cs1296).
- Starvation: Priority, SJF/SPN, SRTF में हो सकता है; FCFS और RR में नहीं (cs1131).
- Priority = waiting time वाला scheduler असल में RR जैसा चलता है

(cs1023).

- Trap: arrival time अलग हो तो उसी क्षण ready queue में कौन है, यह देखो.

⑦ 5. BPSC ऐसे पूछता है (TRE 1.0 / 2.0 / 3.0)

- इस part में TRE का सीधा scheduling सवाल नहीं मिला; TRE में process/thread का सवाल आया: 'Thread shares with other threads its ___' (TRE 3.0, 2024) → code section और data section; thread ID, PC और stack अपने-अपने.

⑧ 6. NET level में ऐसे आता है

- Match the list: MLFQ → queues बदलने का criteria; FCFS → batch; SPN → exponential smoothing; RR → time-slicing (cs0759).
- Process state match (cs0543): Ready→Running = dispatched, Blocked→Ready = event हुआ, Running→Blocked = wait, Running→Ready = preempted.
- Numerical: ऊपर का RR example जैसा.

⑨ 7. याद रखो

- SJF में average waiting सबसे कम; RR में fairness; FCFS में convoy.
- Starvation का इलाज aging.

Critical Section, Deadlock

① 1. आसान भाषा में

- Critical Section (CS) = program का वह हिस्सा जहाँ shared data (जो कई processes के बीच साझा है) बदला जाता है. एक समय में उसमें सिर्फ एक process होना चाहिए.
- CS का हल 3 शर्तें पूरी करे: Mutual Exclusion (एक साथ दो नहीं), Progress (बाहर वाले किसी को रोकें नहीं), Bounded Waiting (कोई

हमेशा इंतज़ार न करे).

- Semaphore = एक counter (integer) जिस पर दो atomic operations चलते हैं: wait()/P (घटाओ, 0 से कम हो तो रुको) और signal()/V (बढ़ाओ, किसी को जगाओ).
- Deadlock = कुछ processes एक-दूसरे के resource का इंतज़ार करते हुए हमेशा के लिए अटक जाएँ.

② 2. रोज़मर्रा का example

- Critical section = एक ही चाबी वाला कमरा (toilet). Semaphore = चाबियों की गिनती.
- Deadlock = दो गाड़ियाँ पतली गली में आमने-सामने; दोनों दूसरे के पीछे हटने का इंतज़ार कर रही हैं.

③ 3. step by step

- Deadlock की 4 शर्तें (चारों साथ चाहिए): Mutual exclusion, Hold and wait, No preemption, Circular wait.
- Deadlock से बचाव: Prevention (किसी एक शर्त को तोड़ो; wait-die scheme), Avoidance (Banker's algorithm: safe state में ही resource दो), Detection (wait-for graph), Recovery (process kill करना सबसे आसान, resource preempt, roll back).
- Safe state = कोई क्रम (safe sequence) हो जिसमें हर process को उसकी max ज़रूरत मिल जाए. Need = Max − Allocated.
- Semaphore का हिसाब: value = start − (P की गिनती) + (V की गिनती).

④ Worked example (safe state)

- 13 tape drives; P1, P2, P3 की max = 11, 5, 8; allocated = 6, 3, 2.

```
Free = 13 - 11 = 2
Need: P1=5, P2=2, P3=6
P2 (2<=2) -> free 5
P1 (5<=5) -> free 11
P3 (6<=11) -> safe: P2 P1 P3
```

5 4. Exam Point + Trap

- Semaphore के operations = wait और signal (cs1219).
- Semaphore numerical: 10 − 12 + x = 7 → x = 9 (cs1289).
- Deadlock हटाने का सबसे आसान = process kill (cs0720).
- Trap: 'Processes outside CS may block others' CS solution की शर्त नहीं; यह उसके विरुद्ध है.
- Unsafe state हमेशा deadlock नहीं होता, पर deadlock का खतरा रहता है.

6 5. BPSC ऐसे पूछता है (TRE 1.0 / 2.0 / 3.0)

- इस part में TRE का सीधा सवाल नहीं मिला. आगे के part में synchronization के और सवाल आएँगे.

7 6. NET level में ऐसे आता है

- Match: Timeout ordering→Thomas write rule, Prevention→Wait-die, Detection→Wait-for graph, Recovery→Roll back (cs0801).
- Deadlock उदाहरण: P1 के पास S, P2 के पास Q, दोनों को दूसरे का चाहिए → Deadlock.
- Safe state numerical (ऊपर).
- Distributed system कथन: user को दूसरी site पर log on नहीं करना पड़ता (transparency).

8 7. याद रखो

- CS = 3 शर्तें; deadlock = 4 शर्तें.
- Safe sequence खोजो = Need ≤ Free वाला process पहले.

Memory Management

① 1. आसान भाषा में

- Program चलने के लिए RAM (Random Access Memory) में होना चाहिए. कई programs एक साथ चलते हैं, इसलिए OS memory बाँटता है.
- Contiguous allocation: हर process को memory में एक सटा हुआ टुकड़ा. External fragmentation = process के बीच के छोटे बेकार छेद. Internal fragmentation = process को दिए block के अंदर बचा बेकार हिस्सा. इलाज: Compaction (processes खिसकाकर सारी खाली जगह एक block).
- Paging: logical memory को बराबर pages में और RAM को उसी size के frames में बाँटते हैं. Page table बताती है कौन-सा page किस frame में. External fragmentation नहीं, पर अंतिम page में internal fragmentation.
- Segmentation: program को logical हिस्सों (code, data, stack) में बाँटते हैं. हर segment के लिए base (RAM में शुरू) और limit (लंबाई).

② 2. रोज़मर्रा का example

- Paging = किताब के बराबर पन्ने; हर पन्ना किसी भी खाली शेल्फ़ खाने में रखा जा सकता है.
- Segmentation = किताब के chapters; हर chapter की लंबाई अलग.
- Compaction = अलमारी के कपड़े एक तरफ खिसकाकर बाकी जगह खाली करना.

③ 3. step by step

- Placement: First fit (पहला जो बड़ा हो), Best fit (सबसे छोटा जो फिट हो), Worst fit (सबसे बड़ा).
- Address: Logical address = page number (p) + offset (d). Physical = frame number × page size + d.
- Offset bits = log2(page size). Page number bits = logical address bits − offset bits.
- TLB (Translation Lookaside Buffer) = page→frame की जल्दी की

छोटी associative cache (सब entries एक साथ compare).

- EAT (Effective Access Time) = h × (T + M) + (1 − h) × (T + 2M); h = hit ratio, T = TLB time, M = memory time.
- TLB tag bits = VPN bits − set-index bits; sets = entries ÷ ways.

④ Worked example (EAT)

- h = 70%, T = 30 ns, M = 100 ns. EAT?

```
Hit = 30 + 100 = 130
Miss = 30 + 100 + 100 = 230
EAT = 0.7x130 + 0.3x230
    = 91 + 69 = 160 ns
```

⑤ 4. Exam Point + Trap

- Page = logical का टुकड़ा; frame = physical का (TRE 3.0).
- Fragmentation memory management technique नहीं; समस्या है (TRE 3.0 NOT सवाल).
- Page offset bits सिर्फ page size से: 4 KB → 12 bits (TRE 3.0). Physical memory size frame number तय करती है.
- Segment base = physical memory में शुरुआती address (TRE 3.0).
- TLB associative होता है (TRE 3.0, cs0719).
- Trap: miss पर 2 memory access (page table + data).

⑥ 5. BPSC ऐसे पूछता है (TRE 1.0 / 2.0 / 3.0)

- Definition: 'What is a page?' / 'What is a segment base?' → ऊपर के points.
- NOT type: 'NOT a memory management technique' → Fragmentation.
- Numerical: 'page size 4 KB, offset bits?' → 12. 'Total memory swapped?' (key A 2 GB, logic 4 GB).
- Compaction: 'processes सटाकर free memory एक block' → Compaction (TRE 1.0).

- More than one of the above: जब एक से ज़्यादा option सही हों (TRE 3.0 file systems, file operations).

7 6. NET level में ऐसे आता है

- Numerical: EAT (cs0758: 160 ns; cs1130: 147 ns).
- TLB tag bits (cs0658): 16 bits.
- Match/Statement: TLB associative memory जो सब keys से एक साथ compare करे (cs0719).

8 7. याद रखो

- Page→Frame; offset = log2(page size).
- EAT में miss पर 2M.

Virtual Memory, Page Replace

1 1. आसान भाषा में

- Virtual memory = RAM से बड़ा program चलाने की तरकीब. सिर्फ ज़रूरी pages RAM में रखते हैं, बाकी disk पर.
- Page fault = ज़रूरी page RAM में नहीं मिला, disk से लाना पड़ेगा.
- RAM भर जाए तो कोई page निकालना पड़ता है. कौन-सा? इसके लिए page replacement algorithms.

2 2. रोज़मर्रा का example

- छोटी मेज़ (RAM) और बड़ी अलमारी (disk). मेज़ पर जगह कम हो तो कोई किताब हटानी पड़ती है. FIFO = सबसे पुरानी रखी हुई हटाओ; LRU = जो सबसे पहले से इस्तेमाल नहीं हुई; Optimal = जो आगे सबसे देर में काम आएगी.

3 3. step by step

- FIFO (First In First Out): सबसे पुराना page निकालो. Belady's anomaly: FIFO में frames बढ़ाने पर faults बढ़ सकते हैं.

- LRU (Least Recently Used): सबसे पुराना use हुआ page. Optimal: जो आगे सबसे देर बाद आएगा; सबसे कम faults, पर असल में लागू नहीं (भविष्य पता नहीं).
- Second Chance: FIFO + reference bit. Bit 1 हो तो एक मौका (bit 0 करके पीछे), bit 0 हो तो निकालो.
- Thrashing: बहुत ज़्यादा paging, CPU असली काम नहीं कर पाता.
- Demand paging page fault time: औसत = p × (modified समय) + (1 − p) × (unmodified समय).

④ Worked example (FIFO)

- String 0 1 2 3 0 1 4 0 1 2 3 4, 3 frames.

```
0F 1F 2F | 3F[1 2 3] 0F[2 3 0] 1F[3 0 1]
4F[0 1 4] 0H 1H 2F[1 4 2] 3F[4 2 3] 4H
Faults = 9; 4 frames □□□ 10 (Belady)
```

⑤ 4. Exam Point + Trap

- Thrashing का कारण = excessive paging (TRE 3.0).
- Initial faults छोड़ने को कहा हो तो घटाओ (cs0811).
- Trap: 'page 5 पहले से load है' कहा हो तो उसका fault मत गिनो (cs0614).

⑥ 5. BPSC ऐसे पूछता है (TRE 1.0 / 2.0 / 3.0)

- Cause type: 'What causes thrashing?' → Excessive paging activity (TRE 3.0).
- Definition type: page, segment base (ऊपर).

⑦ 6. NET level में ऐसे आता है

- FIFO/LRU/Optimal faults गिनना (cs0811, cs1021, cs0614).
- Belady's anomaly (cs1021: 3 frames = 9, 4 frames = 10).

⑧ 7. याद रखो

- LRU और Optimal में Belady नहीं होता; FIFO में हो सकता है.

- Thrashing = paging ज़्यादा, काम कम.

Disk Scheduling, File System

① I. आसान भाषा में

- Hard disk में कई cylinders (गोल tracks) होते हैं. Read/write head को सही cylinder तक ले जाने में समय लगता है (seek time). Disk scheduling तय करती है कि requests किस क्रम में पूरी हों ताकि head का कुल चलना कम हो.
- File system = disk पर files को रखने और ढूँढने का तरीका. उदाहरण: NTFS (New Technology File System), FAT32 (File Allocation Table 32), HFS+ (Hierarchical File System Plus).

② 2. रोज़मर्रा का example

- Lift (elevator) सोचो: कई मंज़िलों से बुलावे आए हैं. FCFS = जिस क्रम में बटन दबे; SSTF = सबसे पास वाली मंज़िल; SCAN = ऊपर जाते हुए सब निपटाओ, फिर नीचे.

③ 3. step by step

- FCFS: आने के क्रम में.
- SSTF (Shortest Seek Time First): सबसे पास वाली request. Starvation हो सकता है.
- SCAN (Elevator): एक दिशा में आखिर तक, फिर उलटा. LOOK: आखिरी request तक ही जाकर उलटा. C-SCAN: एक तरफ ही सर्विस, फिर वापस शुरू में.
- File operations: create, open, read, write, close, delete, rename.
- UNIX file system के हिस्से: boot block, super block (free blocks की list), inode list, data blocks. inode = file की जानकारी. Directory structure = DAG (Directed Acyclic Graph).

④ Worked example (FCFS)

- Head 53; queue 98 183 37 122 14 124 65 67.

```
45+85+146+85+108+110+59+2 = 640 cylinders
```

⑤ 4. Exam Point + Trap

- FCFS total = 640 (TRE 3.0).
- SSTF = 236; SCAN (199 तक जाकर) = 252 (cs0668). LOOK में आखिरी request तक.
- Trap: SCAN disk के आखिर तक जाता है, LOOK नहीं.

⑥ 5. BPSC ऐसे पूछता है (TRE 1.0 / 2.0 / 3.0)

- Numerical: FCFS head movement (TRE 3.0) → 640.
- More than one: file system NTFS/FAT32/HFS+ सब सही; file operation create/delete/rename सब सही.

⑦ 6. NET level में ऐसे आता है

- SSTF/LOOK/SCAN numerical (cs0545, cs0668, cs1356: 119 ms).
- UNIX: super block free list रखता है (cs0813).

⑧ 7. याद रखो

- SSTF पास वाली, SCAN पूरी दिशा, LOOK आखिरी request तक.
- Super block में free blocks की list.

UNIX और Windows facts

① 1. आसान भाषा में

- UNIX एक multi-user OS है. उसमें shell (command line) से काम होता है. Windows 2000 एक Microsoft OS है.
- UNIX के facts: process 0 boot पर बनता है और swapper बनता है; process 1 = init; fork() नया process बनाता है; खत्म हो चुका पर

status न लिया गया = zombie; parent मर गया, child ज़िंदा = orphan.

- Shell script: string compare `test $a = $b`; loop `for n in one two three; do echo -n $n; done`; exit status 0 = सफल.

② 2. रोज़मर्रा का example

- Zombie = कर्मचारी रिटायर हो गया पर उसकी फ़ाइल बंद नहीं हुई (parent ने status नहीं लिया). Orphan = बच्चा जिसके माता-पिता चले गए.

③ 3. step by step (Windows 2000)

- HAL (Hardware Abstraction Layer) hardware के फ़र्क को छुपाता है; processor-dependent code इसी में.
- Kernel: thread scheduling, interrupts, power-failure recovery. Executive: object manager, virtual memory manager, process manager. Win32 subsystem: applications चलाना.
- Thread states: Ready, Standby, Running, Waiting, Transition, Terminated.
- जो हिस्सा portable नहीं: virtual memory management (NET key).

④ 4. Exam Point + Trap

- UNIX directory = DAG; kernel monolithic (cs0618).
- Process 0 init नहीं; init = process 1 (cs0722).
- Windows 2000 में 6 thread states (cs0724).
- User-level thread की कमी: एक thread block तो पूरा process block (cs1177).

⑤ 5. BPSC ऐसे पूछता है (TRE 1.0 / 2.0 / 3.0)

- UNIX/Windows का सीधा TRE सवाल इस part में नहीं मिला.

⑥ 6. NET level में ऐसे आता है

- Match: HAL, Kernel, Executive, Win32 (cs0617).

- 'Which is NOT true' statement type (cs0618, cs0722, cs0611).

7. याद रखो

- Zombie = खत्म पर status नहीं लिया. HAL = hardware छुपाता है.

Numericals — और practice

Practice 1: EAT (Effective Access Time)

- TLB (Translation Lookaside Buffer) = 20 ns, memory = 100 ns, hit ratio = 80%. EAT?

```
Hit = 20+100 = 120
Miss = 20+100+100 = 220
EAT = 0.8x120 + 0.2x220 = 96+44 = 140 ns
```

Practice 2: FCFS disk

- Head 50; queue 82 170 43 140 24 16 190. कुल head movement?

```
|50-82|=32, |82-170|=88, |170-43|=127,
|43-140|=97, |140-24|=116, |24-16|=8, |16-190|=174
□□□ = 642 cylinders
```

Practice 3: Round Robin

- Burst P1 = 5, P2 = 3, P3 = 1, सब time 0, quantum 2. Average TAT और WT?

```
Gantt: P1 0-2, P2 2-4, P3 4-5, P1 5-7, P2 7-8, P1 8-9
Completion P1=9, P2=8, P3=5 -> TAT avg = 22/3 = 7.33
WT = TAT - Burst: 4, 5, 4 -> avg 13/3 = 4.33
```

Practice 4: FIFO और LRU

- String 7 0 1 2 0 3 0 4, 3 frames. दोनों के faults?

```
FIFO: 7F 0F 1F 2F 0H 3F 0F 4F = 7 faults
LRU : 7F 0F 1F 2F 0H 3F 0H 4F = 6 faults
```

5 Practice 5: Safe state

- 12 drives; max P1=10, P2=4, P3=9; allocated 5, 2, 2. Safe क्रम?

```
Free = 12-9 = 3; Need: P1=5, P2=2, P3=7
P2 (2<=3) -> free 5; P1 (5<=5) -> free 10; P3 (7<=10)
Safe sequence: P2 P1 P3
```

6 Practice 6: Offset bits

- Page size 8 KB, virtual address 32 bits. Offset और page-number bits?

```
8 KB = 2^13 -> offset = 13 bits
Page number = 32 - 13 = 19 bits
```

Abbreviations — Full Forms

1 OS Part 2

- OS — Operating System
- CPU — Central Processing Unit
- RAM — Random Access Memory
- FCFS — First Come First Serve
- SJF — Shortest Job First
- SRTF — Shortest Remaining Time First
- SPN — Shortest Process Next
- RR — Round Robin
- MLFQ — Multilevel Feedback Queue
- CFS — Completely Fair Scheduler
- TAT — Turn Around Time
- WT — Waiting Time
- CS — Critical Section
- I/O — Input/Output
- TLB — Translation Lookaside Buffer

- EAT — Effective Access Time
- VPN — Virtual Page Number
- FIFO — First In First Out
- LRU — Least Recently Used
- SSTF — Shortest Seek Time First
- SCAN/LOOK/C-SCAN — Disk arm scheduling algorithms
- NTFS — New Technology File System
- FAT — File Allocation Table
- HFS+ — Hierarchical File System Plus
- DAG — Directed Acyclic Graph
- HAL — Hardware Abstraction Layer
- DLL — Dynamic Link Library
- DMA — Direct Memory Access
- PC — Program Counter
- KB/GB — Kilobyte/Gigabyte

Questions + हर सवाल के नीचे Explanation

Q1. [BPSC TRE 1.0 (2023)]

_____ is used to shift processes so they are contiguous, and all free memory is in one block.

(A) Fragmentation
(B) Compaction
(C) External Fragmentation
(D) More than one of the above
(E) None of the above

हर **option** समझो:

(A) Fragmentation = memory का बेकार टुकड़ों में बँट जाना, यह खुद एक समस्या है
(B) ✔ सही: Compaction processes को खिसकाकर सारी खाली memory एक block बना देता है
(C) External fragmentation समस्या का नाम है, इलाज नहीं
(D) एक ही option सही है, इसलिए 'More than one' गलत
(E) Compaction सही है, इसलिए 'None' गलत

Ans: (B) Compaction

Memory में processes के बीच-बीच में छोटे खाली छेद (holes) रह जाते हैं. Compaction सारे processes को एक तरफ खिसकाकर खाली जगह को एक बड़े block में जोड़ देता है. (जैसे अलमारी में बिखरे कपड़ों को एक तरफ करके दूसरी तरफ खाली जगह बनाना.)

Q2. [BPSC TRE 3.0 (2024)]

Which of the following is NOT a memory management technique used by operating systems?

(A) Paging
(B) Segmentation
(C) Fragmentation
(D) More than one of the above
(E) None of the above

हर **option** समझो:

(A) Paging = memory को बराबर टुकड़ों (pages) में बाँटने की तकनीक है
(B) Segmentation = program को logical हिस्सों में बाँटने की तकनीक है
(C) ✔ सही: Fragmentation तकनीक नहीं, एक problem (बर्बाद memory) है
(D) Paging और segmentation दोनों सही technique हैं, NOT वाला एक ही है
(E) Fragmentation option C technique नहीं है, इसलिए 'None' गलत

Ans: (C) Fragmentation

सवाल 'NOT' वाला है: जो technique नहीं है वह चुनना है. Paging और Segmentation memory बाँटने के तरीके हैं. Fragmentation उनसे पैदा होने वाली बर्बादी है.

Q3. [BPSC TRE 3.0 (2024)]

Which of the following is a file system commonly used in operating systems?

(A) NTFS
(B) FAT32
(C) HFS+
(D) More than one of the above
(E) None of the above

हर **option** समझो:

(A) ✔ सही: NTFS (New Technology File System) Windows का file system है
(B) ✔ सही: FAT32 (File Allocation Table 32) Windows और pen drive में चलता है
(C) ✔ सही: HFS+ (Hierarchical File System Plus) Apple macOS का है
(D) ✔ सही: A, B, C तीनों सही हैं, इसलिए 'More than one'
(E) तीनों file system हैं, इसलिए 'None' गलत

Ans: (D) More than one of the above

तीनों (NTFS, FAT32, HFS+) असली file systems हैं. जब एक से ज़्यादा option सही हों तो 'More than one of the above' चुनते हैं. BPSC में ऐसे सवाल बार-बार आते हैं.

Q4. [BPSC TRE 3.0 (2024)]

What is a page in paging?

(A) A fixed-size block of virtual memory
(B) A fixed-size block of physical memory
(C) A type of memory allocation
(D) More than one of the above
(E) None of the above

हर **option** समझो:

(A) ✔ सही: Page = virtual (logical) memory का fixed-size block
(B) Physical memory के fixed block को page नहीं, frame कहते हैं
(C) Page 'allocation का प्रकार' नहीं, memory का टुकड़ा है
(D) B और C गलत हैं, सिर्फ A सही, इसलिए 'More than one' गलत
(E) A सही है, इसलिए 'None' गलत

Ans: (A) A fixed-size block of virtual memory

Paging में program की logical memory को बराबर टुकड़ों में काटते हैं: इन्हें page कहते हैं. RAM (Random Access Memory) को उसी size के टुकड़ों में काटते हैं: इन्हें frame कहते हैं. Page → Frame में रखा जाता है.

Q5. [BPSC TRE 3.0 (2024)]

In a system with a page size of 4 KB and a physical address space of 64 GB, what is the number of bits required for the page offset?

(A) 10 bits
(B) 12 bits
(C) 14 bits
(D) More than one of the above
(E) None of the above

हर **option** समझो:

(A) 10 bits से सिर्फ 2^10 = 1 KB (Kilobyte) के bytes गिन सकते हैं, 4 KB के नहीं
(B) ✔ सही: 4 KB = 2^12 bytes, इसलिए offset = 12 bits
(C) 14 bits से 16 KB चाहिए, page 4 KB है
(D) एक ही option सही है, इसलिए 'More than one' गलत
(E) 12 bits सही है, इसलिए 'None' गलत

Ans: (B) 12 bits

Page offset बताता है कि page के अंदर कौन-सा byte है. इसलिए offset bits = log2(page size).

Step-by-step Solution:

1. Formula: Page offset bits = log2(Page size)
2. Page size = 4 KB = 4 × 1024 = 2^2 × 2^10 = 2^12 bytes
3. Offset bits = log2(2^12) = 12
4. Answer (B) 12 bits
5. Trap: 64 GB (Gigabyte) physical memory देख कर घबराओ मत. वह सिर्फ frame number (36 − 12 = 24 bits) तय करता है, offset नहीं.

Q6. [BPSC TRE 3.0 (2024)]

What is a segment base in segmentation?

(A) The starting address of a segment in physical memory
(B) The starting address of a segment in virtual memory
(C) The ending address of a segment in physical memory
(D) More than one of the above
(E) None of the above

हर **option** समझो:

(A) ✔ सही: Segment base = segment की physical memory में शुरुआती address
(B) Virtual memory का address segment का offset होता है, base नहीं
(C) आखिरी address base नहीं, base + limit से निकलता है
(D) सिर्फ A सही है, इसलिए 'More than one' गलत
(E) A सही है, इसलिए 'None' गलत

Ans: (A) The starting address of a segment in physical memory

Segmentation में हर segment के लिए दो चीज़ें रखी जाती हैं: Base (RAM में शुरू कहाँ) और Limit (कितना लंबा). Physical address = Base + offset, बशर्ते offset < Limit.

Q7. [BPSC TRE 3.0 (2024)]

What causes thrashing to occur?

(A) Excessive paging activity
(B) Insufficient disk space
(C) Hardware failures
(D) More than one of the above
(E) None of the above

हर **option** समझो:

(A) ✔ सही: बहुत ज़्यादा paging activity (बार-बार page in/out) = thrashing
(B) Disk space कम होना thrashing का कारण नहीं है
(C) Hardware failure अलग समस्या है
(D) सिर्फ A सही, इसलिए 'More than one' गलत
(E) A सही है, इसलिए 'None' गलत

Ans: (A) Excessive paging activity

जब processes को ज़रूरी frames नहीं मिलते तो वे बार-बार page fault करते हैं. CPU (Central Processing Unit) का सारा समय pages अंदर-बाहर करने में जाता है और असली काम रुक जाता है. इसे thrashing कहते हैं.

Q8. [BPSC TRE 3.0 (2024)]

Consider a system with a total physical memory of 8 GB and total virtual memory of 16 GB. If the system is using 6 GB of physical memory and 10 GB of virtual memory, how much memory is being swapped out?

(A) 2 GB
(B) 4 GB
(C) 6 GB
(D) More than one of the above
(E) None of the above

हर **option** समझो:

(A) ✔ सही (BPSC key): 2 GB — official key यही है
(B) logic से 10 GB − 6 GB = 4 GB swapped out होना चाहिए
(C) 6 GB physical memory में है, swapped नहीं
(D) इस सवाल में एक ही option key है
(E) key A है, इसलिए 'None' गलत

Ans: (A) 2 GB

BPSC key: A (2 GB); logic से B (4 GB) (कारण: जितना virtual memory use हो रही है उसमें से जो physical में नहीं है वह swap area में है = 10 – 6 = 4 GB). Exam में BPSC की key (A) ही

लिखो, पर concept याद रखो.

Step-by-step Solution:

1. Virtual memory in use = 10 GB
2. Physical memory in use = 6 GB
3. Swapped out (logic) = 10 − 6 = 4 GB → option B
4. BPSC official key = A (2 GB), इसलिए exam में A mark करना
5. Trap: यह सवाल doubtful है; ऐसे सवाल पर key पर टिको

Q9. [BPSC TRE 3.0 (2024)]

How is the TLB typically organized?

(A) As a direct-mapped cache
(B) As an associative cache
(C) As a set-associative cache
(D) More than one of the above
(E) None of the above

हर **option** समझो:

(A) Direct-mapped में हर page की एक ही जगह होती है, TLB ऐसा नहीं
(B) ✔ सही: TLB associative (सब entries एक साथ compare) cache है
(C) Set-associative भी होता है, पर BPSC key 'associative' है
(D) एक ही option सही, इसलिए 'More than one' गलत
(E) B सही है, इसलिए 'None' गलत

Ans: (B) As an associative cache

TLB (Translation Lookaside Buffer) page→frame की जल्दी याद रखने वाली छोटी memory है. इसमें सारी entries एक साथ compare होती हैं, इसलिए इसे associative memory कहते हैं.

Q10. [BPSC TRE 3.0 (2024)]

Which of the following is a common file operation?

(A) Create
(B) Delete
(C) Rename
(D) More than one of the above
(E) None of the above

हर **option** समझो:

(A) ✔ सही: Create (file बनाना) एक file operation है
(B) ✔ सही: Delete (file हटाना) एक file operation है
(C) ✔ सही: Rename (नाम बदलना) एक file operation है
(D) ✔ सही: A, B, C तीनों सही, इसलिए 'More than one'
(E) तीनों सही हैं, इसलिए 'None' गलत

Ans: (D) More than one of the above

File के common operations: Create, Open, Read, Write, Close, Delete, Rename,

Seek. तीनों option सही हैं, इसलिए D.

Q11. [BPSC TRE 3.0 (2024)]

Consider a disk queue with requests for I/O to blocks on cylinders. 98 183 37 122 14 124 65 67 considering FCFS (First Come First Served) scheduling, the total number of head movements is, if the disk head is initially at 53 is?

(A) 640
(B) 620
(C) 630
(D) More than one of the above
(E) None of the above

हर **option** समझो:

(A) ✔ सही: FCFS (First Come First Served) में कुल 640 cylinders
(B) 620 गलत जोड़ है
(C) 630 गलत जोड़ है
(D) एक ही option सही है, इसलिए 'More than one' गलत
(E) 640 सही है, इसलिए 'None' गलत

Ans: (A) 640

FCFS में disk head requests को आने के क्रम में ही पूरा करता है. हर कदम का फ़र्क जोड़ो.

Step-by-step Solution:

1. Head = 53, queue = 98, 183, 37, 122, 14, 124, 65, 67
2. 53→98 = 45
3. 98→183 = 85
4. 183→37 = 146
5. 37→122 = 85
6. 122→14 = 108
7. 14→124 = 110
8. 124→65 = 59
9. 65→67 = 2
10. कुल = 45+85+146+85+108+110+59+2 = 640 □ (A)
11. Trap: हर फ़र्क positive (|a−b|) लो.

Q12. [BPSC TRE 3.0 (2024)]

Thread shares with other threads belonging to the same process its

(A) thread ID
(B) program counter
(C) code section and data section
(D) More than one of the above
(E) None of the above

हर **option** समझो:

(A) Thread ID हर thread का अपना होता है
(B) Program counter (PC) हर thread का अपना होता है
(C) ✔ सही: code section और data section सब threads share करते हैं
(D) सिर्फ C सही है, इसलिए 'More than one' गलत
(E) C सही है, इसलिए 'None' गलत

Ans: (C) code section and data section

एक process के सारे threads code, data और open files share करते हैं. पर हर thread का अपना ID, program counter, registers और stack होता है.

Q13. [UGC NET Jun 2014 (Paper 3)]

Consider a program that consists of 8 pages (from 0 to 7) and we have 4 page frames in the physical memory for the pages. The page reference string is : 1 2 3 2 5 6 3 4 6 3 7 3 1 5 3 6 3 4 2 4 3 4 5 1 The number of page faults in LRU and optimal page replacement algorithms are respectively (without including initial page faults to fill available page frames with pages) :

(A) 9 and 6
(B) 10 and 7
(C) 9 and 7
(D) 10 and 6

हर **option** समझो:

(A) 9 और 6 में दोनों गलत
(B) ✔ सही: LRU = 10 और Optimal = 7
(C) 9 और 7: LRU गलत
(D) 10 और 6: Optimal गलत

Ans: (B) 10 and 7

LRU (Least Recently Used) = सबसे पुराना use हुआ page हटाओ. Optimal = जो आगे सबसे देर में use होगा उसे हटाओ.

Step-by-step Solution:

1. Reference: 1 2 3 2 5 6 3 4 6 3 7 3 1 5 3 6 3 4 2 4 3 4 5 1, 4 frames
2. शुरू के 4 faults (1,2,3,5) गिनने नहीं हैं
3. LRU: कुल 14 faults − 4 = 10
4. Optimal: कुल 11 faults − 4 = 7
5. Answer (B) 10 और 7
6. Trap: initial faults घटाना है.

Q14. [UGC NET Jan 2017]

In a paging system, it takes 30 ns to search translation Look-a-side Buffer (TLB) and 90 ns to access the main memory. If the TLB hit ratio is 70%, the effective memory access time is :

(A) 48ns
(B) 147ns
(C) 120ns
(D) 84ns

हर **option** समझो:

(A) 48 ns गलत
(B) ✔ सही: 147 ns
(C) 120 ns सिर्फ hit-time है
(D) 84 ns सिर्फ hit का हिस्सा है

Ans: (B) 147ns

TLB hit पर TLB + memory. Miss पर TLB + page table + memory.

Step-by-step Solution:

1. h = 0.7, T = 30, M = 90
2. Hit = 30 + 90 = 120
3. Miss = 30 + 90 + 90 = 210
4. EAT = 0.7×120 + 0.3×210 = 84 + 63 = 147 ns → (B)

Q15. [UGC NET Jul 2016]

Consider the reference string 0 1 2 3 0 1 4 0 1 2 3 4 If FIFO page replacement algorithm is used, then the number of page faults with three page frames and four page frames are

(A) 10, 9
(B) 9, 9
(C) 10, 10
(D) 9, 10

हर **option** समझो:

(A) 10, 9 उल्टा है
(B) 9, 9 गलत है
(C) 10, 10 गलत है
(D) ✔ सही: 3 frames में 9 और 4 frames में 10 (Belady's anomaly)

Ans: (D) 9, 10

FIFO में frames बढ़ाने पर faults कम होने चाहिए पर यहाँ बढ़ गए (9 □ 10). इसे Belady's anomaly कहते हैं.

Step-by-step Solution:

1. Reference: 0 1 2 3 0 1 4 0 1 2 3 4
2. 3 frames FIFO: faults = 0,1,2,3,0,1,4,2,3,4 → 9
3. 4 frames FIFO: faults = 0,1,2,3,4,0,1,2,3,4 → 10
4. Answer (D): 9, 10
5. Trap: ज़्यादा frames = कम faults हमेशा नहीं; FIFO में उल्टा हो सकता है.

Q16. [UGC NET Dec 2014 (Paper 3)]

An operating system has 13 tape drives. There are three processes P1, P2 & P3. Maximum requirement of P1 is 11 tape drives, P2 is 5 tape drives and P3 is 8 tape drives. Currently, P1 is allocated 6 tape drives, P2 is allocated 3 tape drives and P3 is allocated 2 tape drives. Which of the following sequences represent a safe state ?

(A) P2 P1 P3
(B) P2 P3 P1
(C) P1 P2 P3
(D) P1 P3 P2

हर **option** समझो:

(A) ✔ सही: P2 P1 P3 safe sequence है
(B) P2 के बाद P3 को 6 चाहिए पर free 5 है
(C) P1 को 5 चाहिए पर पहले free 2 ही है
(D) P1 पहले नहीं चल सकता

Ans: (A) P2 P1 P3

Safe state = ऐसा क्रम जिसमें हर process को उसकी maximum ज़रूरत मिल जाए. Banker's algorithm की सोच.

Step-by-step Solution:

1. Total 13, allocated = 6+3+2 = 11, free = 2
2. Need = Max − Allocated: P1 = 5, P2 = 2, P3 = 6
3. P2 (need 2 ≤ free 2) चले □ free = 2+3 = 5
4. P1 (need 5 ≤ 5) चले □ free = 5+6 = 11
5. P3 (need 6 ≤ 11) चले □ safe
6. Safe sequence P2 P1 P3 → (A)

Q17. [UGC NET Jul 2016]

A scheduling Algorithm assigns priority proportional to the waiting time of a process. Every process starts with priority zero (lowest priority). The scheduler reevaluates the process priority for every 'T' time units and decides next process to be scheduled. If the process have no I/O operations and all arrive at time zero, then the scheduler implements

(A) Priority scheduling
(B) Round Robin Scheduling
(C) Shortest Job First
(D) FCFS

हर **option** समझो:

(A) Priority scheduling में priority fixed होती है
(B) ✔ सही: Round Robin जैसा व्यवहार (जो ज़्यादा इंतज़ार करे उसे CPU)
(C) SJF में burst length देखी जाती है
(D) FCFS में preempt नहीं होता, यहाँ हर T पर दोबारा चुनाव

Ans: (B) Round Robin Scheduling

Priority = waiting time. हर T पर सबसे ज़्यादा wait वाला चलता है, फिर बाकी का wait बढ़ता है. इससे सब बारी-बारी चलते हैं = Round Robin.

Q18. [UGC NET Jan 2017]

Which of the following scheduling algorithms may cause starvation ? a. First-come-first-served b. Round Robin c. Priority d. Shortest process next e. Shortest remaining time first

(A) a, c and e
(B) c, d and e
(C) b, d and e
(D) b, c and d

हर **option** समझो:

(A) a (FCFS) में starvation नहीं होता
(B) ✔ सही: Priority, SPN (Shortest Process Next) और SRTF (Shortest Remaining Time First) में starvation हो सकता है
(C) Round Robin में सबको बारी मिलती है
(D) b (RR) भी नहीं होता

Ans: (B) c, d and e

जिस algorithm में छोटे या ऊँचे priority वालों को बार-बार आगे रखा जाए, वहाँ लंबे/कम priority वाले हमेशा इंतज़ार करते रह सकते हैं (starvation). इलाज = aging.

Q19. [UGC NET Jul 2018]

In which of the following scheduling criteria, context switching will never take place ?

(A) ROUND ROBIN
(B) Preemptive SJF
(C) Non-preemptive SJF
(D) Preemptive priority

हर **option** समझो:

(A) Round Robin में time quantum पर context switch होता है
(B) Preemptive SJF में छोटा job आने पर switch होता है
(C) ✔ सही: non-preemptive SJF में process पूरा होने तक चलता है
(D) Preemptive priority में higher priority आने पर switch होता है

Ans: (C) Non-preemptive SJF

Context switch = CPU को एक process से दूसरे को देना. Non-preemptive में process बीच में नहीं रुकता, इसलिए context switch नहीं होता.

Q20. [UGC NET Nov 2017]

Two atomic operations permissible on Semaphores are

(A) wait, stop
(B) wait, hold
(C) hold, signal
(D) wait, signal

हर **option** समझो:

(A) 'stop' semaphore operation नहीं है
(B) 'hold' operation नहीं है
(C) hold और signal: hold कोई semaphore operation नहीं है
(D) ✔ सही: wait (P) और signal (V)

Ans: (D) wait, signal

Semaphore के दो atomic operations: wait() यानी P (घटाओ/रुको) और signal() यानी V (बढ़ाओ/जगाओ).

Q21. [UGC NET Sep 2013]

Match the following:

List – I List - II

Process state transition Reason for transition

a Ready→ Running i. Request made by the process is satisfied or an event for which it was waiting occurs.

b Blocked→ Ready ii. Process wishes to wait for some action by another process.

c Running→ Blocked iii. The process is dispatched.

d Running→ Ready iv. The process is preempted.

Codes:

a b c d

(A) iii i ii iv
(B) iv i iii ii
(C) iv iii i ii
(D) iii iii ii i

हर **option** समझो:

(A) ✔ Ready→Running का मतलब process को CPU मिला = dispatched (iii)
(B) ✔ सही: a-iii, b-i, c-ii, d-iv
(C) Preempted (iv) Running→Ready के लिए है, Ready→Running के लिए नहीं
(D) b-iii, c-ii, d-i मेल नहीं खाते

Ans: (A) iii i ii iv

Process के states: Ready (CPU का इंतज़ार), Running (CPU पर), Blocked (I/O का इंतज़ार).
Ready→Running = dispatch; Running→Ready = preempt (time खत्म);
Running→Blocked = process खुद wait करे; Blocked→Ready = इंतज़ार की चीज़ मिल गई.

Q22. [UGC NET Sep 2013]

Consider the input/output (I/O) requests made at different instants of time directed at a hypothetical disk having 200 tracks as given in the following table:

Serial No. 1 2 3 4 5

Track No. 12 85 40 100 75

Time of arrival 65 80 110 100 175

Assume that :

Current head position is at track no. 65

Direction of last movement is towards higher numbered tracks

Current clock time is 160 milliseconds

Head movement time per track is 1 millisecond.

"look" is a variant of "SCAN" diskarm scheduling algorithm. In this algorithm, if no more I/O requests are left in current direction, the disk head reverses its direction. The seek times in Shortest Seek First (SSF) and "look" disk-arm scheduling algorithms respectively are :

(A) 144 and 123 milliseconds
(B) 143 and 123 milliseconds
(C) 149 and 124 milliseconds
(D) 256 and 186 milliseconds

हर **option** समझो:

(A) 144 और 123: SSF का जोड़ 143 है, 144 नहीं
(B) ✔ सही: SSF (Shortest Seek First) = 143 और LOOK = 123 ms
(C) 149 और 124 गलत जोड़ हैं
(D) 256 और 186 बहुत ज़्यादा हैं

Ans: (B) 143 and 123 milliseconds

Head 65 पर, clock 160 ms. तब तक requests 12, 85, 40, 100 आ चुकी हैं और 75 (175 ms पर) रास्ते में आ जाती है. 1 track = 1 ms.

Step-by-step Solution:

1. SSF: हमेशा सबसे पास वाली request. 65→85 = 20 (अब time 180, 75 आ चुकी)
2. 85→75 = 10, 75→100 = 25, 100→40 = 60, 40→12 = 28
3. कुल = 20+10+25+60+28 = 143 ms
4. LOOK (ऊपर की तरफ चल रहा था): 65□85 = 20, 85□100 = 15
5. फिर उलटा: 100□75 = 25, 75□40 = 35, 40□12 = 28
6. कुल = 20+15+25+35+28 = 123 ms → Answer (B)
7. Trap: 75 (arrival 175) को भूलना नहीं, क्योंकि 85 तक पहुँचते-पहुँचते वह आ चुकी है.

Q23. [UGC NET Jun 2014 (Paper 3)]

_____ maintains the list of free disk blocks in the Unix file system.

(A) I-node
(B) Boot block
(C) Super block
(D) File allocation table

हर **option** समझो:

(A) I-node एक file की जानकारी रखता है
(B) Boot block में boot program होता है
(C) ✔ सही: Super block में free blocks की list होती है
(D) FAT (File Allocation Table) Windows का है

Ans: (C) Super block

UNIX file system में super block पूरे file system की जानकारी रखता है: size, free blocks और free inodes की list.

Q24. [UGC NET Jul 2018]

At a particular time of computation, the value of a counting semaphore is 10. Then 12 P operations and "x" V operations were performed on this semaphore. If the final value of semaphore is 7, x will be :

(A) 8
(B) 9
(C) 10
(D) 11

हर **option** समझो:

(A) 8 लेने पर value 6 आएगी
(B) ✔ सही: $x = 9 \rightarrow 10 - 12 + 9 = 7$
(C) 10 लेने पर value 8 आएगी
(D) 11 लेने पर value 9 आएगी

Ans: (B) 9

हर P (wait) value 1 घटाता है और हर V (signal) 1 बढ़ाता है.

Step-by-step Solution:

1. Start = 10
2. 12 P operations: $10 - 12 = -2$
3. x V operations: $-2 + x$
4. $-2 + x = 7 \rightarrow x = 9 \rightarrow$ (B)

Q25. [UGC NET Sep 2013 (Paper 3)]

Which of the following statements is not correct with reference to distributed systems?

(A) Distributed system represents a global view of the network and considers it as a virtual uniprocessor system by controlling and managing resources across the network on all the sites.

(B) Distributed system is built on bare machine, not an add-on to the existing software.

(C) In a distributed system, kernel provides smallest possible set of services on which other services are built. This kernel is called microkernel. Open servers provide other services and access to shared resources.

(D) In a distributed system, if a user wants to run the program on other nodes or share the resources on remote sites due to their beneficial aspects, user has to log on to that site.

हर **option** समझो:

(A) ✔ सही कथन: distributed system पूरे network को एक virtual uniprocessor जैसा दिखाता है

(B) ✔ सही कथन: यह bare machine पर बनता है, add-on नहीं

(C) ✔ सही कथन: microkernel सबसे कम services देता है

(D) ✔ सही: यह कथन गलत है — distributed OS में user को दूसरी site पर log on नहीं करना पड़ता

Ans: (D) In a distributed system, if a user wants to run the program on other nodes or share the resources on remote sites due to their beneficial aspects, user has to log on to that site.

Distributed OS में user को पता भी नहीं चलता कि program कहाँ चल रहा है (transparency). इसलिए 'दूसरी site पर log on करना पड़ता है' कथन गलत है. यह network OS की बात है.

Q26. [UGC NET Jan 2017 (Paper 3)]

One of the disadvantages of user level threads compared to Kernel level threads is

(A) If a user level thread of a process executes a system call, all threads in that process are blocked.

(B) Scheduling is application dependent.

(C) Thread switching doesn’t require kernel mode privileges.

(D) The library procedures invoked for thread management in user level threads are local procedures.

हर **option** समझो:

(A) ✔ सही: एक user thread system call करे तो पूरा process (सारे threads) block हो जाता है

(B) Scheduling application के हाथ में होना advantage है

(C) kernel mode के बिना switching advantage है
(D) Local procedures advantage है

Ans: (A) If a user level thread of a process executes a system call, all threads in that process are blocked.

User-level thread को kernel जानता ही नहीं. इसलिए एक thread block हो तो kernel पूरे process को block कर देता है. यही बड़ा नुकसान है.

Q27. [UGC NET Jun 2019]

Consider a disk system with 100 cylinders. The requests to access the cylinders occur in the following sequence: 4,34,10,7,19,73,2,15,6,20
Assuming that the head is current at cylinder 50, what is the time taken to satisfy all requests if it takes 1ms to move from the cylinder to adjacent one and the shortest seek time first policy is used?

(A) 357 ms
(B) 238 ms
(C) 276 ms
(D) 119 ms

हर **option** समझो:

(A) 357 ms बहुत ज़्यादा है
(B) 238 ms गलत है
(C) 276 ms गलत है
(D) ✔ सही: 119 ms

Ans: (D) 119 ms

SSTF में हमेशा सबसे पास वाली request. 1 cylinder = 1 ms.

Step-by-step Solution:

1. Head = 50, requests: 4,34,10,7,19,73,2,15,6,20
2. 50→34 = 16
3. 34→20 = 14, 20→19 = 1, 19→15 = 4
4. 15→10 = 5, 10→7 = 3, 7→6 = 1
5. 6→4 = 2, 4→2 = 2, 2→73 = 71
6. कुल = 16+14+1+4+5+3+1+2+2+71 = 119 ms → (D)

Q28. [UGC NET Sep 2013 (Paper 3)]

Consider a main memory with 3 page frames for the following page reference string:
5, 4, 3, 2, 1, 4, 3, 5, 4, 3, 4, 1, 4
Assuming that the execution of process is initiated after loading page 5 in memory, the number of page faults in FIFO and second chance replacement respectively are

(A) 8 and 9
(B) 10 and 11
(C) 7 and 9
(D) 9 and 8

हर **option** समझो:

(A) 8 और 9: FIFO के 9 सही हैं पर Second Chance 8 है, 9 नहीं
(B) 10 और 11: ये page 5 का initial load भी गिनने से आते हैं
(C) 7 और 9: दोनों गिनती गलत हैं
(D) ✔ सही: FIFO (First In First Out) में 9 और Second Chance में 8 faults

Ans: (D) 9 and 8

सवाल कहता है कि process page 5 load होने के बाद शुरू हुआ, यानी पहली बार 5 का fault नहीं गिनना.

Step-by-step Solution:

1. Reference: 5,4,3,2,1,4,3,5,4,3,4,1,4 ; 3 frames
2. FIFO: 5 पहले से load है (गिनती नहीं). 4 □ F1 [5,4], 3 → F2 [5,4,3]
3. 2 → F3 (5 निकला) [4,3,2]; 1 □ F4 (4 निकला) [3,2,1]
4. 4 → F5 (3 निकला) [2,1,4]; 3 □ F6 (2 निकला) [1,4,3]; 5 □ F7 (1 निकला) [4,3,5]
5. 4,3,4 → hits; 1 → F8 (4 निकला) [3,5,1]; 4 □ F9 (3 निकला) □ FIFO = 9
6. Second Chance (हर page के साथ reference bit; bit 1 हो तो एक मौका देकर 0 करो): ऊपर की तरह F7 तक वही. फिर 4 और 3 के hits से उनके bits = 1
7. 1 आने पर: 4 (bit 1) बचा, 3 (bit 1) बचा, 5 (bit 0) निकला □ F8. फिर 4 hit. कुल = 8
8. Answer (D) 9 और 8
9. Trap: शुरू का page 5 fault गिन लोगे तो 10 और 9 आएगा.

Q29. [UGC NET Sep 2013 (Paper 3)]

The portion of Windows 2000 operating system which is not portable is

(A) Processor management
(B) User interface
(C) Device management
(D) Virtual memory management

हर **option** समझो:

(A) Processor management portable रखने के लिए HAL (Hardware Abstraction Layer)

का सहारा लेता है

(B) User interface hardware पर निर्भर नहीं

(C) Device management drivers के ज़रिये बदला जा सकता है

(D) ✔ सही: Virtual memory management processor पर निर्भर है, इसलिए portable नहीं

Ans: (D) Virtual memory management

Windows 2000 का ज़्यादातर code portable है. जो हिस्सा processor के hardware से सीधे जुड़ा है, यानी virtual memory management, portable नहीं माना गया (NET key).

Q30. **[UGC NET Sep 2013 (Paper 3)]**

Match the following for Windows Operating System:

(a) Hardware abstraction layer (i) Starting all processes, emulation of different operating systems, security functions, transform character based applications to graphical representation.

(b) Kernel (ii) Export a virtual memory interface, support for symmetric multiprocessing administration, details of mapping memory, configuring I/O buses, setting up DMA.

(c) Executive (iii) Thread scheduling, interrupt and exception handling, recovery after power failure.

(d) Win32 subsystem (iv) Object manager, virtual memory manager, process manager, plug and- play and power manager.

Codes:

(a) (b) (c) (d)

(A) (i) (iii) (ii) (iv)

(B) (iv) (iii) (ii) (i)

(C) (ii) (iii) (iv) (i)

(D) (iii) (ii) (i) (iv)

हर **option** समझो:

(A) a-i गलत: HAL का काम hardware छुपाना है, processes शुरू करना नहीं

(B) b-iii ठीक है पर बाकी मेल गलत

(C) ✔ सही: HAL-ii, Kernel-iii, Executive-iv, Win32-i

(D) a-iii गलत: thread scheduling Kernel का काम है

Ans: (C) (ii) (iii) (iv) (i)

HAL (ii): hardware के details छुपाता है, I/O buses, DMA (Direct Memory Access). Kernel (iii): thread scheduling, interrupts, power-failure recovery. Executive (iv): object, virtual memory और process manager. Win32 subsystem (i): applications चलाना.

Q31. [UGC NET Sep 2013 (Paper 3)]

Which of the following statements is not true for UNIX Operating System?

(A) Major strength of UNIX Operating System is its open standards which enabled large number of organizations ranging from academician to industries to participate in its development.

(B) UNIX kernel uses modules with well specified interfaces and provides advantages like simplified testing and maintenance of kernel. It has better extensibility as the kernel is not monolithic.

(C) UNIX is kernel based operating system with two main components viz. process management subsystem and file management subsystem.

(D) All devices are represented as files which simplify the management of I/O devices and files. The directories structure used is directed acyclic graph.

हर **option** समझो:

(A) ✔ सही कथन: UNIX की ताक़त open standards है

(B) ✔ सही: यह कथन गलत है — UNIX kernel monolithic है, modular/non-monolithic नहीं

(C) ✔ सही कथन: UNIX में process management और file management दो main subsystems हैं

(D) ✔ सही कथन: UNIX में हर device एक file है और directory structure DAG (Directed Acyclic Graph) है

Ans: (B) UNIX kernel uses modules with well specified interfaces and provides advantages like simplified testing and maintenance of kernel. It has better extensibility as the kernel is not monolithic.

UNIX का kernel monolithic (एक बड़ा पूरा block) माना जाता है. इसलिए 'kernel monolithic नहीं है' कहने वाला कथन गलत है.

Q32. [UGC NET Dec 2013]

The virtual address generated by a CPU is 32 bits. The Translation Look-aside Buffer (TLB) can hold total 64 page table entries and a 4-way set associative (i.e. with 4- cache lines in the set). The page size is 4 KB. The minimum size of TLB tag is

(A) 12 bits
(B) 15 bits
(C) 16 bits
(D) 20 bits

हर **option** समझो:

(A) 12 bits तो page offset है, tag नहीं
(B) 15 bits गलत है
(C) ✔ सही: tag = 20 − 4 = 16 bits
(D) 20 bits पूरा virtual page number है, पर set index के 4 bits घटाने हैं

Ans: (C) 16 bits

TLB में page number (VPN, Virtual Page Number) का एक हिस्सा set चुनता है (index) और बाकी हिस्सा tag बनता है.

Step-by-step Solution:

1. Virtual address = 32 bits, page size 4 KB = 2^12 → offset = 12 bits
2. Virtual page number (VPN) = 32 − 12 = 20 bits
3. TLB entries = 64, 4-way → sets = 64 / 4 = 16 = 2^4 → index = 4 bits
4. Tag = VPN − index = 20 − 4 = 16 bits → (C)
5. Trap: offset के 12 bits tag में नहीं जाते.

Q33. [UGC NET Dec 2013]

Linux operating system uses

(A) Affinity Scheduling
(B) Fair Preemptive Scheduling
(C) Hand Shaking
(D) Highest Penalty Ratio Next

हर **option** समझो:

(A) Affinity scheduling processor से process जोड़ने की बात है
(B) ✔ सही: Linux Fair Preemptive Scheduling (CFS — Completely Fair Scheduler) use करता है
(C) Hand shaking communication का शब्द है
(D) HRRN (Highest Response Ratio Next) अलग algorithm है, 'penalty ratio' नहीं

Ans: (B) Fair Preemptive Scheduling

Linux का scheduler सब processes को बराबर CPU समय देने की कोशिश करता है और ज़रूरत पर

process को बीच में रोक देता है (preemptive).

Q34. [UGC NET Dec 2013]

Consider a disk queue with request for input/output to block on cylinders
98, 183, 37, 122, 14, 124, 65, 67
in that order. Assume that disk head is initially positioned at cylinder 53 and moving towards cylinder number 0. The total number of head movements using Shortest Seek Time First (SSTF) and SCAN algorithms are respectively

(A) 236 and 252 cylinders
(B) 640 and 236 cylinders
(C) 235 and 640 cylinders
(D) 235 and 252 cylinders

हर **option** समझो:

(A) ✔ सही (key): SSTF (Shortest Seek Time First) = 236 और SCAN = 252 (disk के आखिर 199 तक)
(B) 640 और 236: 640 तो FCFS का total है
(C) 235 और 640: दोनों गलत हैं
(D) 235 और 252: SSTF का जोड़ 236 है, 235 नहीं

Ans: (A) 236 and 252 cylinders

Head 53 पर, 0 की तरफ जा रहा है. SSTF हमेशा सबसे पास वाली request लेता है.

Step-by-step Solution:

1. SSTF: 53→65 = 12, 65→67 = 2, 67→37 = 30, 37→14 = 23
2. 14→98 = 84, 98→122 = 24, 122→124 = 2, 124→183 = 59
3. कुल = 12+2+30+23+84+24+2+59 = 236
4. SCAN (0 की तरफ): 53□0 = 53, फिर 0□199 (disk का आखिर, 200 cylinders) = 199
5. कुल = 53+199 = 252 (key यही लेती है) □ (A)
6. Trap: SCAN disk के आखिर तक जाता है; 183 पर रुकने वाला LOOK होता है (तब 236).

Q35. [UGC NET Dec 2013 (Paper 3)]

In Unix, how do you check that two given strings a and b are equal ?

(A) test $a –eq $b
(B) test $a –equal $b
(C) test $a = $b
(D) Sh –C test $a = = $b

हर **option** समझो:

(A) -eq numbers के लिए है, strings के लिए नहीं

(B) -equal कोई operator नहीं है
(C) ✔ सही: test $a = $b strings को compare करता है
(D) '==' और sh -C वाला रूप गलत है

Ans: (C) test $a = $b

UNIX shell में strings की बराबरी = (single equals) से जाँचते हैं, numbers की -eq से.

Q36. [UGC NET Dec 2013 (Paper 3)]

In windows 2000 operating system all the processor-dependent code is isolated in a dynamic link library called

(A) NTFS file system
(B) Hardware abstraction layer
(C) Microkernel
(D) Process Manager

हर **option** समझो:

(A) NTFS file system है, processor code नहीं
(B) ✔ सही: HAL (Hardware Abstraction Layer) में processor-dependent सारा code है
(C) Microkernel scheduling करता है
(D) Process Manager processes सँभालता है

Ans: (B) Hardware abstraction layer

Windows 2000 में hardware के फ़र्क को HAL (एक DLL, Dynamic Link Library) छुपाता है ताकि बाकी OS बदलना न पड़े.

Q37. [UGC NET Dec 2013 (Paper 3)]

Translation Look-aside Buffer (TLB) is

(A) a cache-memory in which item to be searched is compared one-by-one with the keys.
(B) a cache-memory in which item to be searched is compared with all the keys simultaneously.
(C) an associative memory in which item to be searched is compared one-by-one with the keys.
(D) an associative memory in which item to be searched is compared with all the keys simultaneously.

हर **option** समझो:

(A) Cache नहीं, associative memory कहते हैं और compare सब साथ होता है
(B) Cache + one-by-one गलत है
(C) Associative memory है, पर compare एक-एक करके नहीं होता
(D) ✔ सही: associative memory, जिसमें item सब keys से एक साथ compare होता है

Ans: (D) an associative memory in which item to be searched is compared with all the keys simultaneously.

TLB associative memory है: दी गई key सभी entries से एक साथ मिलाई जाती है, इसलिए यह तेज़ है.

Q38. [UGC NET Dec 2013 (Paper 3)]

Simplest way of deadlock recovery is

(A) Roll back
(B) Preempt resource
(C) Lock one of the processes
(D) Kill one of the processes

हर **option** समझो:

(A) Roll back में state save करना पड़ता है, यह आसान नहीं
(B) Resource preempt करना कठिन और complicated है
(C) 'Lock process' recovery नहीं है
(D) ✔ सही: process को kill कर देना सबसे आसान तरीका है

Ans: (D) Kill one of the processes

Deadlock हटाने के तरीके: process kill करना (सबसे आसान), resource preempt करना, roll back करना. Kill करने में बस process हटाओ और resources free हो जाते हैं.

Q39. [UGC NET Dec 2013 (Paper 3)]

The directory structure used in Unix file system is called

(A) Hierarchical directory
(B) Tree structured directory
(C) Directed acyclic graph
(D) Graph structured directory

हर **option** समझो:

(A) Hierarchical (tree) में links नहीं होते
(B) Tree-structured में share नहीं होता
(C) ✔ सही: UNIX में links की वजह से directory structure DAG (Directed Acyclic Graph) है
(D) Graph-structured में cycle हो सकता है, UNIX में नहीं

Ans: (C) Directed acyclic graph

UNIX में एक file के कई नाम (links) हो सकते हैं, पर cycle नहीं बनने देते. इसलिए structure DAG है.

Q40. [UGC NET Dec 2013 (Paper 3)]

Which statement is not true about process O in the Unix operating system ?

(A) Process O is called init process.
(B) Process O is not created by fork system call.
(C) After forking process 1, process O becomes swapper process.
(D) Process O is a special process created when system boots.

हर **option** समझो:

(A) ✔ सही: यह कथन गलत है — process 0 init नहीं है, init process 1 है
(B) ✔ सही कथन: process 0 fork से नहीं बनता
(C) ✔ सही कथन: process 1 बनाने के बाद process 0 swapper बन जाता है
(D) ✔ सही कथन: process 0 boot के समय बनता है

Ans: (A) Process O is called init process.

UNIX में process 0 boot के समय बनता है (fork से नहीं). वह process 1 (init) बनाकर swapper बन जाता है. 'Process 0 = init' कहना गलत है.

Q41. [UGC NET Jul 2016 (Paper 3)]

In UNIX, processes that have finished execution but have not yet had their status collected are known as

(A) Sleeping processes
(B) Stopped processes
(C) Zombie processes
(D) Orphan processes

हर **option** समझो:

(A) Sleeping process अभी चल रहा है, खत्म नहीं हुआ
(B) Stopped process रोका गया है
(C) ✔ सही: Zombie = खत्म हो चुका पर parent ने status नहीं लिया
(D) Orphan का parent खत्म हो चुका होता है

Ans: (C) Zombie processes

Child process खत्म हो गया पर parent ने wait() से उसका status नहीं लिया: वह zombie है (process table में अटका). Parent पहले मर जाए तो child orphan बनता है.

Q42. [UGC NET Dec 2013 (Paper 3)]

Possible thread states in Windows 2000 operating system include :

(A) Ready, running and waiting
(B) Ready, standby, running, waiting, transition and terminated
(C) Ready, running, waiting, transition and terminated
(D) Standby, running, transition and terminated

हर **option** समझो:

(A) सिर्फ तीन states बताईं, बाकी छूटीं
(B) ✔ सही: Ready, Standby, Running, Waiting, Transition, Terminated
(C) Standby जोड़ना होगा, Ready भी है
(D) Ready गायब है

Ans: (B) Ready, standby, running, waiting, transition and terminated

Windows 2000 में thread की 6 states होती हैं: Ready, Standby, Running, Waiting, Transition, Terminated.

Q43. [UGC NET Jun 2014]

In a paged memory management algorithm, the hit ratio is 70%. If it takes 30 nanoseconds to search Translation Look-aside Buffer (TLB) and 100 nanoseconds (ns) to access memory, the effective memory access time is

(A) 91 ns
(B) 69 ns
(C) 200 ns
(D) 160 ns

हर **option** समझो:

(A) 91 ns सिर्फ hit का हिस्सा है
(B) 69 ns सिर्फ miss का हिस्सा है
(C) 200 ns miss का time भी नहीं है
(D) ✔ सही: 160 ns

Ans: (D) 160 ns

Hit पर TLB search + memory access. Miss पर TLB search + page table access + memory access.

Step-by-step Solution:

1. Formula: EAT (Effective Access Time) = h × (T + M) + (1 − h) × (T + 2M)
2. h = 0.7, T (TLB) = 30, M (memory) = 100
3. Hit = 30 + 100 = 130
4. Miss = 30 + 100 + 100 = 230
5. EAT = 0.7×130 + 0.3×230 = 91 + 69 = 160 ns → (D)

6. Trap: 91 और 69 अलग-अलग हिस्से हैं, जोड़ना है.

Q44. [UGC NET Jun 2014]

Match the following : List – I List – II a. Multilevel feedback queue i. Time-slicing b. FCFS ii. Criteria to move processes between queues c. Shortest process next iii. Batch processing d. Round robin scheduling iv. Exponential smoothening Codes : a b c d

(A) i iii ii iv
(B) iv iii ii i
(C) iii i iv i
(D) ii iii iv i

हर **option** समझो:

(A) a-i गलत: MLFQ (Multilevel Feedback Queue) में queues बदलने का criteria है
(B) b-iii ठीक पर बाकी गलत
(C) c-iv ठीक पर a गलत
(D) ✔ सही: MLFQ-ii, FCFS-iii, SPN-iv, RR-i

Ans: (D) ii iii iv i

MLFQ में process queues के बीच खिसकता है (criteria to move). FCFS batch में चलता है. SPN (Shortest Process Next) में अगला burst अंदाज़े से (exponential smoothing) लगाते हैं. RR (Round Robin) में time-slicing होती है.

Q45. [UGC NET Jun 2014]

Which of the following commands will output “onetwothree” ?

(A) for val; do echo-n $val; done < one two three
(B) for one two three; do echo-n-; done
(C) for n in one two three; do echo-n $n; done
(D) `for n in one two three {echo –n $ n}`

हर **option** समझो:

(A) 'for val' और redirection गलत syntax
(B) 'for one two three' गलत syntax
(C) ✔ सही: for n in one two three; do echo -n $n; done
(D) brace वाला syntax shell में नहीं चलता

Ans: (C) for n in one two three; do echo-n $n; done

Shell loop का syntax: for var in list; do command; done. echo -n newline नहीं छापता, इसलिए output onetwothree एक लाइन में आता है.

Q46. [UGC NET Jun 2014]

Which of the following is the correct value returned to the operating system upon the successful completion of a program ?

(A) 0
(B) 1
(C) – 1
(D) Program do not return a value.

हर **option** समझो:

(A) ✔ सही: 0 = success (सफलता)
(B) 1 आम तौर पर error का code है
(C) -1 भी error है
(D) Program value return करता है (exit status)

Ans: (A) 0

UNIX/C++ में main() सफल होने पर 0 लौटाता है. कोई भी non-zero error माना जाता है.

Q47. [UGC NET Jun 2014 (Paper 3)]

Match the following : List – I List – II a. Timeout ordering protocol i. Wait for graph b. Deadlock prevention ii. Roll back c. Deadlock detection iii. Wait-die scheme d. Deadlock recovery iv. Thomas Write Rule Codes : a b c d

(A) iv iii i ii
(B) iii ii iv i
(C) ii i iv iii
(D) iii i iv iii

हर **option** समझो:

(A) ✔ सही: Timeout-ordering→Thomas Write Rule, Prevention→Wait-die, Detection→Wait-for graph, Recovery→Roll back
(B) a-iii गलत
(C) a-ii गलत
(D) d-iii दो बार आया, गलत

Ans: (A) iv iii i ii

Wait-die deadlock prevention की scheme है. Wait-for graph से deadlock detect होता है. Roll back recovery है. Thomas write rule timestamp ordering protocol में आता है.

Answer Key

Q1 - B	Q2 - C	Q3 - D	Q4 - A	Q5 - B
Q6 - A	Q7 - A	Q8 - A	Q9 - B	Q10 - D
Q11 - A	Q12 - C	Q13 - B	Q14 - B	Q15 - D
Q16 - A	Q17 - B	Q18 - B	Q19 - C	Q20 - D
Q21 - A	Q22 - B	Q23 - C	Q24 - B	Q25 - D
Q26 - A	Q27 - D	Q28 - D	Q29 - D	Q30 - C
Q31 - B	Q32 - C	Q33 - B	Q34 - A	Q35 - C
Q36 - B	Q37 - D	Q38 - D	Q39 - C	Q40 - A
Q41 - C	Q42 - B	Q43 - D	Q44 - D	Q45 - C
Q46 - A	Q47 - A			